

**MedSupplyDrones** is a drone-based medication delivery service designed for emergency situations where traditional pharmacy dispatch may be unavailable. Each drone flight follows a fixed pattern: depart from a service point, collect medication from multiple pick-up points, deliver to one or more drop-off locations, and return to the original service point. The system calculates the optimal route for a given list of orders from a medication management system. Service points are distributed across the geographical area, each housing multiple drones. Drones are dispatched based on their attributes including capacity, temperature-controlled storage (cooling/heating), maximum range, cost per move, and availability schedule. The system matches orders to compatible drones, navigates around restricted no-fly zones, minimises total cost, and coordinates multiple drones when orders have conflicting requirements.

## Application Requirements:

### Non-Functional Requirements

| Requirement                                                                                                                                                      | Type                     | Level/Priority             | Test Approach                                                                                                                  | Limitations                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Performance - Calculate delivery routes efficiently with pathfinding completing within reasonable time limits. API responses fast enough for real-time dispatch. | Non-Functional (Quality) | System Testing / High      | Performance benchmarking with varying input sizes; load testing under concurrent requests; measure response times against SLAs | Test environment may not replicate production load; difficult to simulate true emergency conditions             |
| Scalability - Handle multiple concurrent route calculations and support growing fleet without performance degradation.                                           | Non-Functional (Quality) | System Testing / High      | Stress testing with increasing concurrent users; capacity testing with growing data sets                                       | Upper bounds of scalability hard to determine without production data; resource constraints in test environment |
| Reliability - Highly available, consistent correct routes. Failed calculations handled gracefully with appropriate error responses.                              | Non-Functional (Quality) | System Testing / High      | Fault injection testing; chaos engineering; verify error handling paths; measure MTTF over extended runs                       | True availability requires long-term production monitoring; cannot simulate all failure modes                   |
| Maintainability - Modular, well-structured codebase separating concerns (pathfinding, drone selection, API handling).                                            | Non-Functional (Quality) | N/A (Code Review) / Medium | Static code analysis; code review against design principles; measure cyclomatic complexity and coupling metrics                | Subjective assessment: metrics don't capture all maintainability aspects                                        |
| Interoperability - Integrate with external services using standard REST protocols and JSON data formats.                                                         | Non-Functional (Quality) | Integration Testing / High | Contract testing against API specifications; schema validation testing; mock external services                                 | External API behaviour may change; mocks may not reflect real service behaviour                                 |

|                                                                                                                  |                          |                                       |                                                                                                                       |                                                                                                        |
|------------------------------------------------------------------------------------------------------------------|--------------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Safety - Route planning must strictly respect no-fly zone restrictions; drones never enter restricted areas.     | Non-Functional (Safety)  | Integration/System Testing / Critical | Boundary testing at zone edges; exhaustive path segment validation; property-based testing for zone avoidance         | Cannot test all possible paths; edge cases at zone boundaries may be missed                            |
| Accuracy - Distance calculations, position tracking, and route planning maintain sufficient numerical precision. | Non-Functional (Quality) | Unit Testing / High                   | Boundary value testing at precision limits; comparison against known correct values; floating-point edge case testing | Flat Earth approximation inherent in Pythagorean distance; accumulated rounding errors hard to predict |

## Functional Requirements

| Requirement                                                                                                                           | Type                      | Level/Priority                  | Test Approach                                                                                                         | Limitations                                                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------|---------------------------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Read all dynamic data from the ILP REST service before each calculation. No data should be hard-coded.                                | Functional                | Unit Testing / High             | Mock ILP service responses; verify no hardcoded values via code inspection; test with varying service data            | Cannot guarantee all hardcoded values detected; mock may diverge from real service             |
| Use the ILP REST endpoints for drone information, service points, and restricted areas.                                               | Functional                | Integration Testing / High      | Integration tests against real/stubbed ILP endpoints; verify correct endpoint calls and data parsing                  | External service availability; API changes outside our control                                 |
| Accept orders as JSON via REST endpoints. Orders are the only instruction data passed into the service.                               | Functional                | Integration Testing / High      | Schema validation testing; test valid, malformed, and edge-case JSON inputs; verify rejection of non-JSON             | Cannot anticipate all malformed input variants                                                 |
| Provide endpoints that organise and validate data to find suitable drones based on capacity, cooling/heating, cost, and availability. | Functional                | Unit/Integration Testing / High | Equivalence partitioning for drone attributes; boundary testing for capacity limits; test all constraint combinations | Combinatorial explosion of constraint combinations                                             |
| Assign multiple orders to flights so drone picks up once and delivers to multiple drop-offs before returning to base.                 | Functional                | Unit Testing / High             | Test with varying order combinations; verify single pickup per flight; path validation for return to base             | Optimal assignment is NP-hard; testing optimality is difficult                                 |
| Minimise the number of drones and moves used while respecting all constraints.                                                        | Functional (Optimisation) | System Testing / Medium         | Compare against known optimal solutions for small inputs; regression testing to ensure no degradation                 | Optimal solution unknown for complex inputs; trade-off between optimality and computation time |

|                                                                                                             |                          |                                       |                                                                                                              |                                                                                |
|-------------------------------------------------------------------------------------------------------------|--------------------------|---------------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Ensure each flight starts and ends at the same service point and remains within drone's maximum move limit. | Functional (Correctness) | Unit Testing / High                   | Invariant checking on all generated paths; boundary testing at move limits                                   | Edge cases with exactly max moves may behave differently                       |
| Compute stepwise paths using 0.00015 degree moves, constrained to 16 compass directions.                    | Functional (Correctness) | Unit Testing / High                   | Verify step distances exactly 0.00015°; verify angles match 16 allowed directions; geometric validation      | Floating-point precision may cause minor deviations                            |
| Represent hovering as two consecutive moves at the same coordinate.                                         | Functional (Correctness) | Unit Testing / Medium                 | Test hover detection; verify coordinate equality for hover moves                                             | Simple requirement with limited edge cases                                     |
| Apply closeness checks: location reached if path enters within 0.00015 degrees of target.                   | Functional (Correctness) | Unit Testing / High                   | Boundary value testing at exactly 0.00015°, just inside, just outside; test from multiple approach angles    | Circular vs square closeness regions may cause edge discrepancies              |
| Avoid restricted no-fly zones entirely. Paths must not enter or cross these areas.                          | Functional (Safety)      | Integration/System Testing / Critical | Path segment intersection testing; test paths near zone boundaries; property-based testing with random paths | Cannot exhaustively test all possible paths; complex zone shapes increase risk |
| Calculate total flight cost as: initial cost + final cost + (moves × cost per move).                        | Functional (Correctness) | Unit Testing / High                   | Verify formula implementation with known values; test with zero moves, max moves, varying costs              | Simple calculation with limited edge cases                                     |
| Return cost and move statistics in results for efficiency evaluation.                                       | Functional               | Integration Testing / Medium          | Verify response schema includes all statistics; cross-check reported values against path data                | Dependent on correct path generation                                           |
| Expose a REST API returning JSON with appropriate HTTP status codes.                                        | Functional               | Integration Testing / High            | Test all endpoints for correct status codes (200, 400, 404, 500); verify JSON response format                | Cannot test all possible error conditions                                      |
| For each dispatch request, return computed paths, costs, and summary statistics.                            | Functional               | Integration Testing / High            | End-to-end testing of dispatch flow; verify response completeness and accuracy                               | Dependent on all upstream components working correctly                         |
| Treat locations as (longitude, latitude) in degrees using Pythagorean distance.                             | Functional (Correctness) | Unit Testing / High                   | Verify coordinate order (log, lat); compare distances against manual calculations                            | Pythagorean approximation invalid for large distances                          |

|                                                                                      |                          |                                     |                                                                                                |                                                                              |
|--------------------------------------------------------------------------------------|--------------------------|-------------------------------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Complete calculations within the 30-second time budget.                              | Functional (Performance) | System Testing / High               | Timeout testing with complex inputs; performance profiling; test boundary at ~30s              | Execution time varies with hardware; may pass in test but fail in production |
| Run inside a Docker container with configuration provided via environment variables. | Functional (Deployment)  | System/Integration Testing / Medium | Container deployment testing; verify env var configuration; test with missing/invalid env vars | Container behaviour may differ across environments                           |

## Testing Strategy Evaluation

### Bottom-Up Approach

A bottom-up testing approach is well-suited to the specification of requirements derived from the project behaviour description. This strategy progresses through three levels: unit testing, integration testing, and system testing.

### Unit Testing

Unit level tests ensure that standalone modules work reliably and cover all edge cases, minimising the possibility of unexpected behaviour. Key areas for unit testing include geometry calculations (distance, closeness, point-in-region), direction and step calculations, cost calculations, and drone filtering logic. These are pure functions with predictable outputs, making them ideal for comprehensive unit test coverage.

### Integration Testing

Integration tests build on top of unit tests to verify that component interactions behave as intended and produce desired results. This includes testing the coordination between pathfinding, drone selection, and route planning modules. API endpoint testing verifies correct HTTP status codes (200, 400, 404) and response schema compliance.

### System Testing

System level tests ensure that the system as a whole works reliably and meets high-level operational goals in a simulated production environment. Key areas for system testing include validating the 30-second calculation time budget, ensuring scalability for concurrent route requests, and verifying strict adherence to no-fly zone safety restrictions across entire flight paths. Additionally, these tests confirm the system's ability to minimise total moves and costs while coordinating multiple drones with conflicting requirements. By quantifying qualitative requirements through measurable benchmarks like execution time and Mean-Time-To-Failure (MTTF), system testing ensures the application is robust, deterministic, and ready for deployment within its Docker container.

### Quantifying Qualitative Requirements

Qualitative requirements such as robustness and performance are quantified through measurable benchmarks. These include execution time (ensuring calculations complete within the 30-second budget), relative test code coverage, and in production,

collected statistics such as Mean-Time-To-Failure (MTTF). Determinism is verified by running identical requests multiple times and confirming identical outputs.

### **Extensibility**

Using this bottom-up approach, should system or integration requirements change or evolve, additional functional requirements and their respective unit tests can be easily introduced and integrated without disrupting existing test coverage.

### **Limitations**

A potential limitation is testing against the external ILP REST API, which is outside of our control. Its behaviour and responses cannot be guaranteed to remain consistent, potentially impeding the robustness and reliability of the application. This risk can be partially mitigated through mocking the external service in tests and implementing defensive error handling for unexpected responses.